

Técnicas de Regresión

Pastor E Pérez Estigarribia

28 de mayo de 2026

Regresión lineal: objetivo y notación

Problema: dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^n$, $x_i \in \mathbb{R}^p$, estimar y como combinación lineal:

$$y_i \approx \beta_0 + x_i^\top \beta, \quad \beta \in \mathbb{R}^p.$$

Forma matricial:

$$\mathbf{y} = X\beta + \varepsilon, \quad X \in \mathbb{R}^{n \times p}, \quad \varepsilon \sim (0, \sigma^2 I).$$

Comentario: la formulación asume linealidad en los parámetros; extensible con bases no lineales o features polinómicas.

Estimación OLS (mínimos cuadrados ordinarios)

Criterio: minimizar la suma de cuadrados de los residuos

$$\hat{\beta}_{OLS} = \arg \min_{\beta} \|\mathbf{y} - X\beta\|_2^2.$$

Solución (ecuación normal):

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T \mathbf{y} \quad \text{si } X^T X \text{ invertible.}$$

Alternativas numéricas:

- **QR decomposition:** más estable numéricamente que invertir $X^T X$.
- **SVD:** útil si X es mal condicionada; permite regularización por truncamiento.

Propiedades: estimador insesgado; varianza $\text{Var}(\hat{\beta}) = \sigma^2 (X^T X)^{-1}$.

Problemas de OLS y motivación para regularizar

- **Multicolinealidad:** $X^T X$ mal condicionada $\Rightarrow$ coeficientes con alta varianza.
- **p grande vs n pequeño:** sobreajuste cuando p cercano o mayor que n , (p : predictores o features (columnas) en la matriz X)
- **Selección de variables:** OLS no realiza selección automática.

Idea: añadir penalizaciones sobre β para controlar complejidad y mejorar generalización.

Ridge Regression (L2) — formulación

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2 \}, \quad \lambda \geq 0.$$

Solución cerrada:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Interpretación:

- Penaliza coeficientes grandes; reduce varianza a costa de introducir sesgo.
- Equivalente a MAP con prior gaussiano $\beta \sim \mathcal{N}(0, \tau^2 \mathbf{I})$.

Lasso (L1) — formulación y propiedades

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \{ \|\mathbf{y} - X\beta\|_2^2 + \lambda \|\beta\|_1 \}.$$

Características:

- Penalización L1 induce **esparsidad** (coeficientes exactamente cero).
- No tiene solución cerrada; se resuelve por métodos iterativos (coordinate descent, LARS).
- Útil para selección de variables y modelos interpretables.

$$\hat{\beta}_{\text{EN}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \alpha\lambda\|\beta\|_1 + (1 - \alpha)\lambda\|\beta\|_2^2 \},$$

con $\alpha \in [0, 1]$ controlando mezcla L1/L2. **Ventajas:**

- Combina selección (L1) y estabilidad frente a multicolinealidad (L2).
- Recomendado cuando hay grupos de variables correlacionadas.

Evaluación del rendimiento (regresión)

Métricas comunes

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Validación:

- **Train/Validation/Test** o **k-fold CV** para estimar generalización.
- **Nested CV** si se realiza selección de hiperparámetros.

Criterios de información: AIC, AICc y BIC

AIC (Akaike Information Criterion)

$$AIC = 2k - 2 \log \hat{L},$$

donde k = número de parámetros y $\hat{L}$ = verosimilitud máxima.

AICc (corrección para muestras pequeñas)

$$AICc = AIC + \frac{2k(k+1)}{n-k-1}.$$

BIC (Bayesian Information Criterion)

$$BIC = k \log n - 2 \log \hat{L}.$$

Interpretación: criterios que penalizan complejidad; elegir modelo con menor AIC/AICc/BIC según objetivo (AIC favorece ajuste, BIC penaliza más complejidad).

Cómo calcular AIC/BIC en regresión Gaussiana

Para modelo lineal con errores gaussianos y SSE:

$$\log \hat{L} = -\frac{n}{2} (\log(2\pi\sigma^2) + 1), \quad \hat{\sigma}^2 = \frac{\text{SSE}}{n}.$$

Sustituyendo se obtiene expresiones prácticas para AIC/BIC en términos de SSE y k . **Nota práctica:** cuando n pequeño usar AICc; para p grande comparar también con CV (MSE CV).

Comparación de modelos: AIC/BIC vs CV

- **AIC/BIC:** basados en verosimilitud; rápidos; asumen modelo correctamente especificado.
- **Cross-validation:** estima error predictivo directo (MSE CV); más robusto si el objetivo es minimizar error de predicción.
- **Recomendación práctica:** usar CV para rendimiento predictivo; usar AIC/BIC para selección de parsimonia y modelos interpretables; reportar ambos cuando sea relevante.

Regularización: selección de hiperparámetros

- λ (Ridge/Lasso) y α (ElasticNet) se seleccionan típicamente por **CV** (GridSearchCV, RandomizedSearchCV, o CV específico como LassoCV/ElasticNetCV).
- **Path algorithms** (glmnet) permiten calcular soluciones para una secuencia de λ eficientemente y seleccionar por CV.
- **Práctica:** estandarizar variables (StandardScaler) antes de aplicar penalizaciones.

Regresión polinómica — idea esencial

- Extender la regresión lineal permitiendo términos no lineales en las variables: $y \approx \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d$.
- Mantiene la linealidad en los parámetros β — por eso se resuelve con técnicas de regresión lineal.
- Útil para modelar relaciones suaves no lineales entre predictor y respuesta.

Formulación matricial

$$X_{\text{poly}} = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^d \end{pmatrix}, \quad \mathbf{y} = X_{\text{poly}}\beta + \varepsilon.$$

Nota: la dimensión efectiva del problema crece con d ; conviene controlar complejidad.

Estimación OLS para polinomios

$$\hat{\beta}_{OLS} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}_{\text{poly}}\beta\|_2^2 \Rightarrow \hat{\beta} = (\mathbf{X}_{\text{poly}}^\top \mathbf{X}_{\text{poly}})^{-1} \mathbf{X}_{\text{poly}}^\top \mathbf{y}$$

Comentarios prácticos:

- Usar factorizaciones (QR, SVD) para estabilidad numérica.
- Escalar/centrar x reduce colinealidad entre potencias.

Problemas: sobreajuste y bias-variance

- **Grado bajo (underfitting):** alto sesgo, error en entrenamiento y test.
- **Grado alto (overfitting):** baja sesgo en entrenamiento, alta varianza y mal desempeño fuera de muestra.
- **Trade-off bias-variance:** elegir d que minimice error predictivo (CV).

Selección del grado por validación cruzada

$$\hat{R}_{CV}(d) = \frac{1}{K} \sum_{i=1}^K \text{MSE}(\hat{f}_{-i}^{(d)}, D_i)$$

- Evaluar varios d y elegir $d^* = \arg \min_d \hat{R}_{CV}(d)$.
- Recomendación: usar $K = 5$ o 10 ; en muestras pequeñas considerar LOOCV.
- Para selección de hiperparámetros (grado, λ) usar nested CV si se requiere estimación final no sesgada.

Regularización para polinomios (Ridge / Lasso)

$$\text{Ridge: } \hat{\beta} = \arg \min_{\beta} \{ \|\mathbf{y} - X_{\text{poly}}\beta\|_2^2 + \lambda \|\beta\|_2^2 \}$$

$$\text{Lasso: } \hat{\beta} = \arg \min_{\beta} \{ \|\mathbf{y} - X_{\text{poly}}\beta\|_2^2 + \lambda \|\beta\|_1 \}$$

Ventajas: controlan varianza; Lasso puede inducir esparsidad en coeficientes polinómicos; Ridge estabiliza cuando hay multicolinealidad entre potencias.

Interpretación y práctica con polinomios

- **Efecto de λ :** mayor $\lambda \Rightarrow$ coeficientes más pequeños $\Rightarrow$ curva más suave.
- **Escalado:** siempre estandarizar x antes de generar potencias o usar bases ortogonales.
- **Bases alternativas:** polinomios ortogonales (Legendre, Chebyshev) o splines para mejor comportamiento numérico y extrapolación.

Evaluación del rendimiento (regresión)

Métricas

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}.$$

Validación: usar CV para estimar MSE CV; reportar media y desviación estándar.

Criterios de información para modelos polinómicos

$$AIC = 2k - 2 \log \hat{L}, \quad AICc = AIC + \frac{2k(k+1)}{n-k-1},$$

$$BIC = k \log n - 2 \log \hat{L}.$$

Notas:

- k incluye intercepto y coeficientes efectivos; $\log \hat{L}$ para errores gaussianos se obtiene desde SSE.
- AIC/AICc favorecen ajuste; BIC penaliza más complejidad.
- Complementar AIC/BIC con CV (MSE CV) para decisiones prácticas.

Diagnósticos y visualizaciones recomendadas

- **Curva ajustada vs datos:** scatter de x vs y y curva $\hat{y}(x)$.
- **Residuos vs fitted:** detectar heterocedasticidad y patrones no modelados.
- **Curvas de validación:** error de entrenamiento y validación vs grado d (learning/validation curve).
- **Intervalos de confianza** para la predicción y para la curva media.

- Centrar y escalar x antes de elevar a potencias.
- Preferir `PolynomialFeatures + StandardScaler` en Pipeline.
- Evitar invertir $X^T X$ directamente; usar SVD/QR o implementaciones de librerías.
- Para p grande o d alto usar regularización y/o reducción de dimensionalidad.

Regresión de cuantiles — idea esencial

- La regresión de cuantiles estima la relación entre predictores y un **cuantil condicional** $q \in (0, 1)$ de la variable respuesta, no solo la media.
- Permite modelar la **mediana** ($q = 0,5$) o colas (p.ej. $q = 0,1, 0,9$), útil con heterocedasticidad o asimetría.

Formulación matemática (lineal)

Para un cuantil q , la regresión lineal de cuantiles busca

$$\hat{\beta}_q = \arg \min_{\beta} \sum_{i=1}^n \rho_q(y_i - x_i^\top \beta),$$

donde la **función pinball** (loss) es

$$\rho_q(u) = \begin{cases} q u & u \geq 0, \\ (q - 1) u & u < 0. \end{cases}$$

Comentario: para $q = 0,5$ se obtiene la regresión por la mediana (LAD).

- **Programación lineal:** la minimización del pinball loss se puede formular como un problema lineal y resolverse con solvers LP (simplex, interior-point).
- **Algoritmos iterativos:** métodos basados en re-weighted least squares o subgradientes para grandes conjuntos.
- **Implementaciones:** 'statsmodels.QuantReg' (estimación por LP) y 'scikit-learn' (GLM/QuantileRegressor en versiones recientes).

- **Robustez a outliers:** la mediana ($q=0.5$) es menos sensible que la media.
- **Heterocedasticidad:** permite ver cómo cambia la dispersión condicional con x .
- **Información completa:** estimar varios cuantiles da una visión de la distribución condicional completa.

- **Pinball loss** promedio en validación (métrica natural para cuantiles).
- Comparar con MSE/RMSE de OLS cuando interesa la media; usar CV para estimar error predictivo.
- Visualizaciones: curvas de cuantiles condicionales (gráfica x vs $\hat{q}(x)$) y bandas de incertidumbre por bootstrap.

Comparación rápida: OLS vs Quantile Regression

Aspecto	OLS (media)	Quantile (cuantil)
Objetivo	Minimizar SSE; estima media condicional	Minimizar pinball loss; estima cuantil condicional
Robustez	Sensible a outliers y Más robusta (p.ej. mediana)	
Heterocedasticidad	Ineficiente si varianza no constante	Modela cambios en la dispersión
Interpretación	Efecto promedio	Efectos en distintas partes de la distribución

Implementaciones prácticas (scikit-learn / statsmodels)

- **statsmodels**: QuantReg — API orientada a análisis estadístico; resumen con errores estándar y tests.
- **scikit-learn**: ejemplos y utilidades en `'linear_model' / GLMparacuantiles; verejemplooficialdequantileregression`.
- **Workflow recomendado**: preparar datos, ajustar varios $q(p.ej, 0, 1, 0, 5, 0, 9)$, *validar con CV (pinball loss)*, *visualizar cuantiles condicionales*.

Ejemplo de diagnóstico y visualización

- Graficar observaciones y curvas $\hat{q}(x)$ para varios q .
- Bootstrap para intervalos de confianza de coeficientes por cuantiles.
- Comparar predicciones condicionales (p.ej. percentiles 10–90) para decisiones de riesgo.

- scikit-learn: ejemplo de Quantile regression (documentación de `model_selection` / GLM).
- statsmodels: 'QuantReg' y ejemplos aplicados (Koenker y Hallock references).
- Tutoriales y posts prácticos sobre interpretación y visualización de cuantiles.

Funciones relevantes en scikit-learn (referencia rápida)

- `PolynomialFeatures` — generar potencias y términos cruzados.
- `LinearRegression` — OLS (SVD).
- `Ridge`, `RidgeCV` — Ridge y selección de λ .
- `Lasso`, `LassoCV`, `ElasticNet`, `ElasticNetCV`.
- `Pipeline`, `make_pipeline` — encadenar escalado y modelo.
- `GridSearchCV`, `RandomizedSearchCV` — búsqueda de grado y λ con CV.
- `sklearn.preprocessing.PolynomialFeatures` + `StandardScaler` recomendado.

Ejemplo de flujo práctico (resumen)

- 1 Visualizar datos y sospecha de no linealidad.
- 2 Construir pipeline: `StandardScaler` → `PolynomialFeatures(degree=d)` → `Ridge/Lasso`.
- 3 Seleccionar d y λ por CV (`GridSearchCV` o `ElasticNetCV`).
- 4 Evaluar con CV y en conjunto test independiente; reportar MSE CV y AIC/BIC si interesa parsimonia.
- 5 Revisar residuos y estabilidad de coeficientes.

- Polinomios de grado alto pueden comportarse mal fuera del rango de los datos (extrapolación).
- Para relaciones complejas y localmente variables, prefiera splines o modelos no paramétricos.
- Documentar la elección de grado y regularización; reportar reproducibilidad (semillas, versiones).

glmnet (nota práctica)

- **glmnet** es una implementación eficiente (pathwise coordinate descent) de penalizaciones tipo Elastic Net (Lasso/Ridge) muy usada en R y disponible en Python (bindings).
- Implementa cálculo de $\beta(\lambda)$ a lo largo de una secuencia de λ (regularization path) y selección por CV.
- **Comentario:** en Python se puede usar `sklearn.linear_model.ElasticNetCV` o paquetes que exponen glmnet para obtener rutas y eficiencia en grandes problemas.

Métodos de estimación numérica (resumen)

- **Closed form:** OLS y Ridge (inversa o factorización).
- **Coordinate descent:** eficiente para Lasso/ElasticNet (actualiza coeficiente por coeficiente).
- **LARS (Least Angle Regression):** camino Lasso eficiente para p grande.
- **Gradiente / SGD:** útil para MLE/regularización en datasets muy grandes.
- **CV interno:** para elegir λ y α (GridSearchCV, ElasticNetCV).

Funciones relevantes en scikit-learn (referencia rápida)

- `LinearRegression` — OLS (closed form / SVD).
- `Ridge`, `RidgeCV` — regresión Ridge; `RidgeCV` selecciona λ por CV.
- `Lasso`, `LassoCV` — Lasso y selección automática de λ .
- `ElasticNet`, `ElasticNetCV` — Elastic Net y búsqueda de α , λ .
- `SGDRegressor` — optimización por gradiente estocástico (útil para grandes datasets).
- `GridSearchCV`, `RandomizedSearchCV` — búsqueda de hiperparámetros con CV.
- `statsmodels.api.OLS` — alternativa para estimación OLS con salida estadística (p-values, AIC/BIC).

Buenas prácticas y checklist

- Escalar features antes de penalizar (StandardScaler en Pipeline).
- Evaluar con CV y, si se seleccionan hiperparámetros, usar nested CV para estimación final.
- Comparar modelos con métricas predictivas (MSE CV) y criterios de información (AIC/AICc/BIC) según objetivo.
- Reportar λ seleccionado, error CV, y estabilidad (repetir con distintas semillas si procede).

Ejemplo de flujo práctico (resumen)

- 1 Preprocesado: imputación, escalado, generación de features.
- 2 Baseline: `LinearRegression` (OLS).
- 3 Regularización: `RidgeCV`, `LassoCV`, `ElasticNetCV`.
- 4 Selección: comparar MSE CV, AIC/AICc/BIC y complejidad del modelo.
- 5 Validación final: evaluar modelo elegido en conjunto test independiente.

Referencias y lecturas recomendadas

- Documentación scikit-learn: `linear_model` (LinearRegression, Ridge, Lasso, ElasticNet, SGDRegressor).
- glmnet (R) y bindings Python para rutas de regularización y selección por CV.